Exercise 4: Buckpal with Hexagonal Architecture

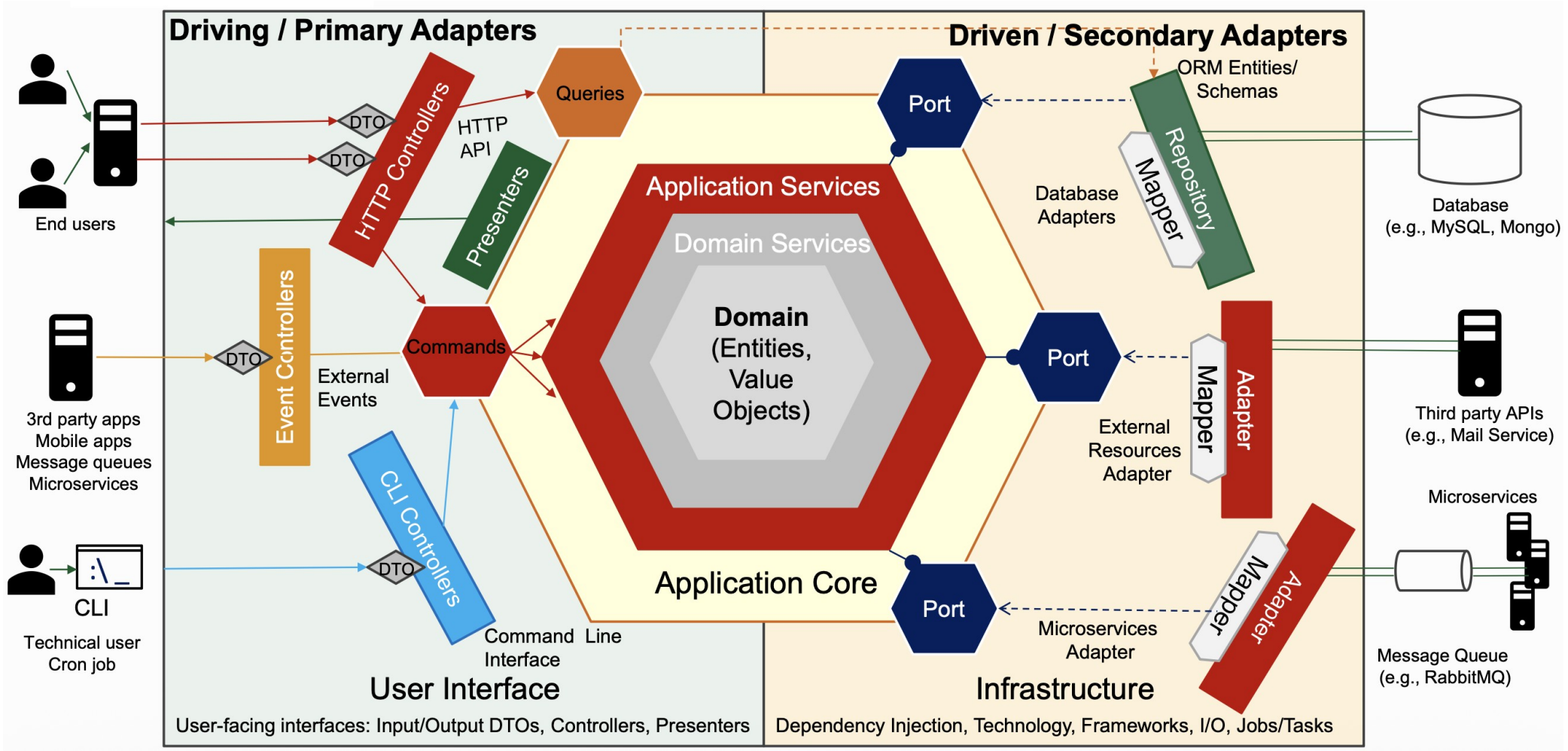
Software Architecture

Winter Semester 2026/27

Tobias Arens M.sc.

swa Software Architecture Teaching and Research Unit | RWTH AACHEN UNIVERSITY

Hexagonal Architecture: Overview



Example Flow

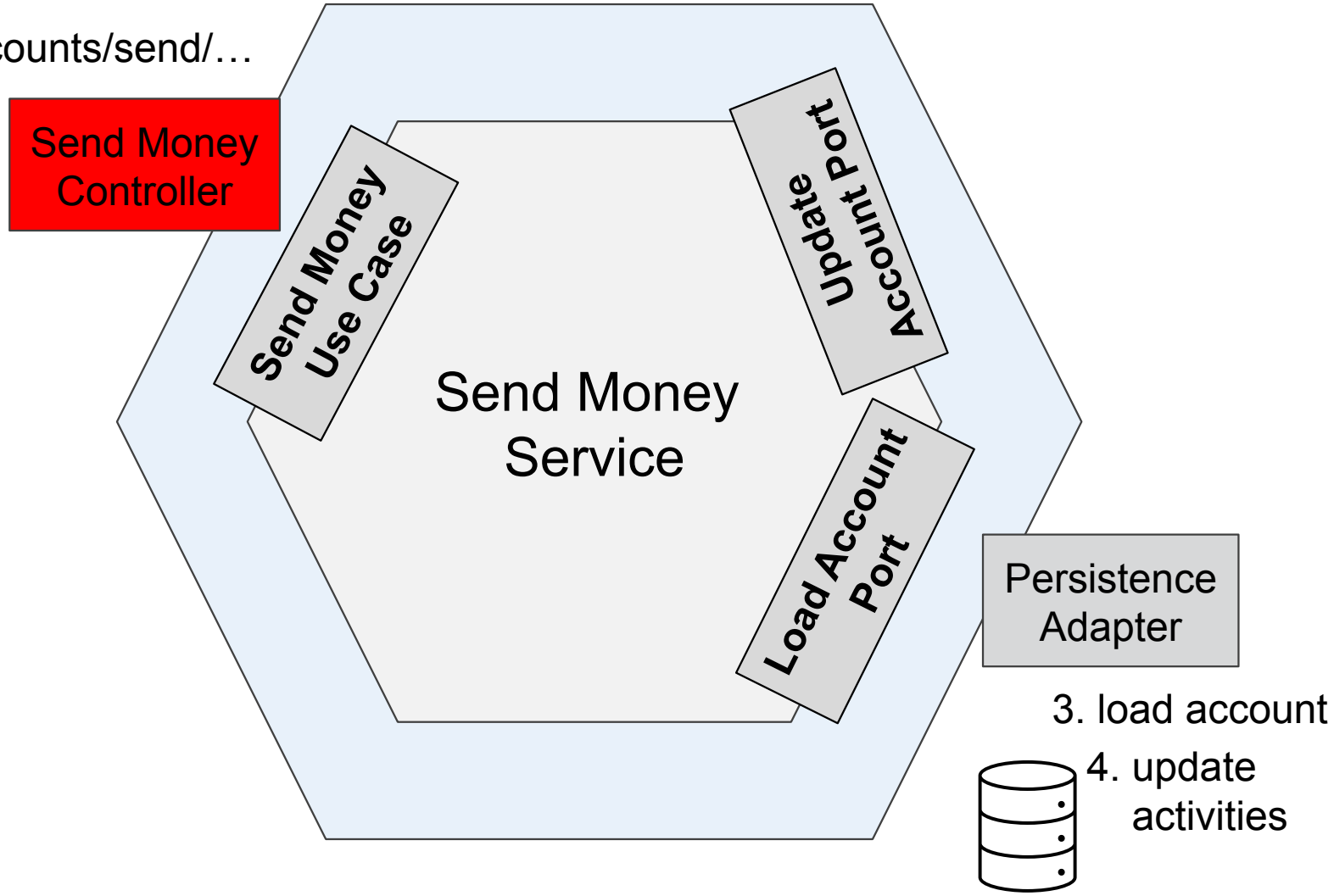
1. send money





Web UI
(frontend)

2. POST: accounts/send/…



Example Controller: HTTP Adapter (Driving Adapter)

```java
@WebAdapter  Tom Hombergs
@RestController
@RequiredArgsConstructor
class SendMoneyController {

    private final SendMoneyUseCase sendMoneyUseCase;

    @PostMapping(path = "/accounts/send/{sourceAccountId}/{targetAccountId}/{amount}")
    void sendMoney(
            @PathVariable("sourceAccountId") Long sourceAccountId,
            @PathVariable("targetAccountId") Long targetAccountId,
            @PathVariable("amount") Long amount) {

        SendMoneyCommand command = new SendMoneyCommand(
                new AccountId(sourceAccountId),
                new AccountId(targetAccountId),
                Money.of(amount));

        sendMoneyUseCase.sendMoney(command);
    }

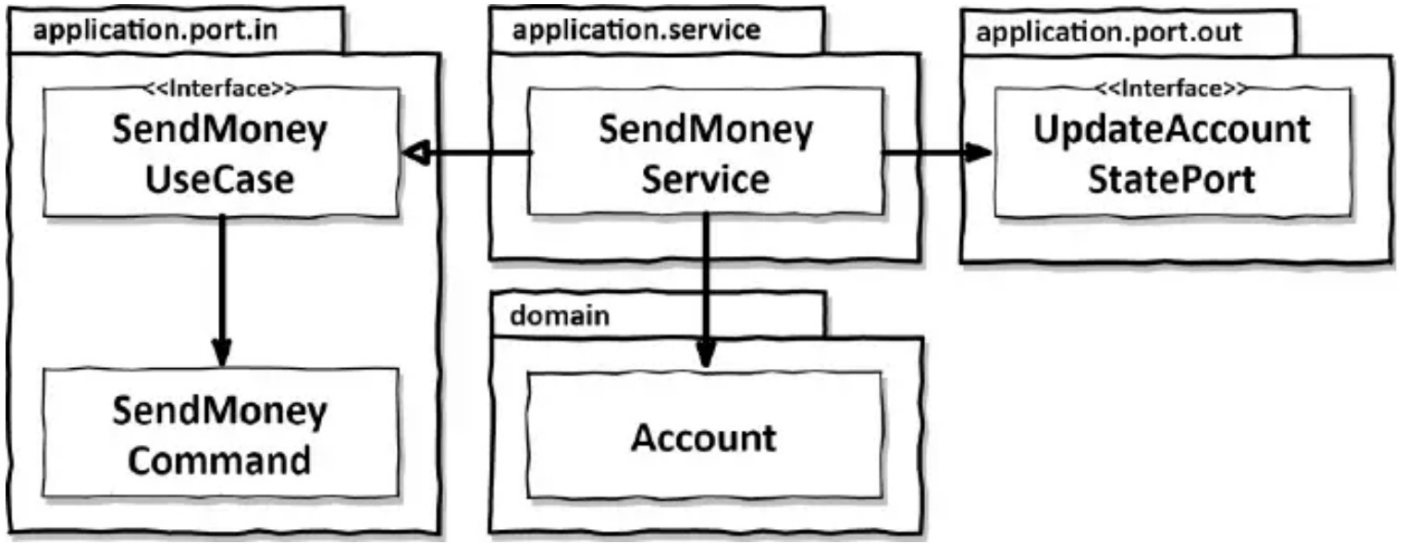
}
```

Responsibilities:
1. Map HTTP request to Java objects
2. Perform authorization checks
3. Validate input (i.e., validate that we can transform the input model of the web adapter to the input model of the use case)
4. Map input to the input model of the use case
5. Call the use case
6. Map output of the use case back to HTTP
7. Return HTTP response

Calling the use case,
the command carries input data and intention
(command → manipulation)

Buckpal: Example Application Layer





Source: Get Your Hands Dirty on Clean Architecture

Source code: https://github.com/thombergs/buckpal

Send Money Use Case

SendMoneyUseCase.java

```java
package io.reflectoring.buckpal.application.port.in;

public interface SendMoneyUseCase {  6 usages  1 implementation  Tom Hombergs

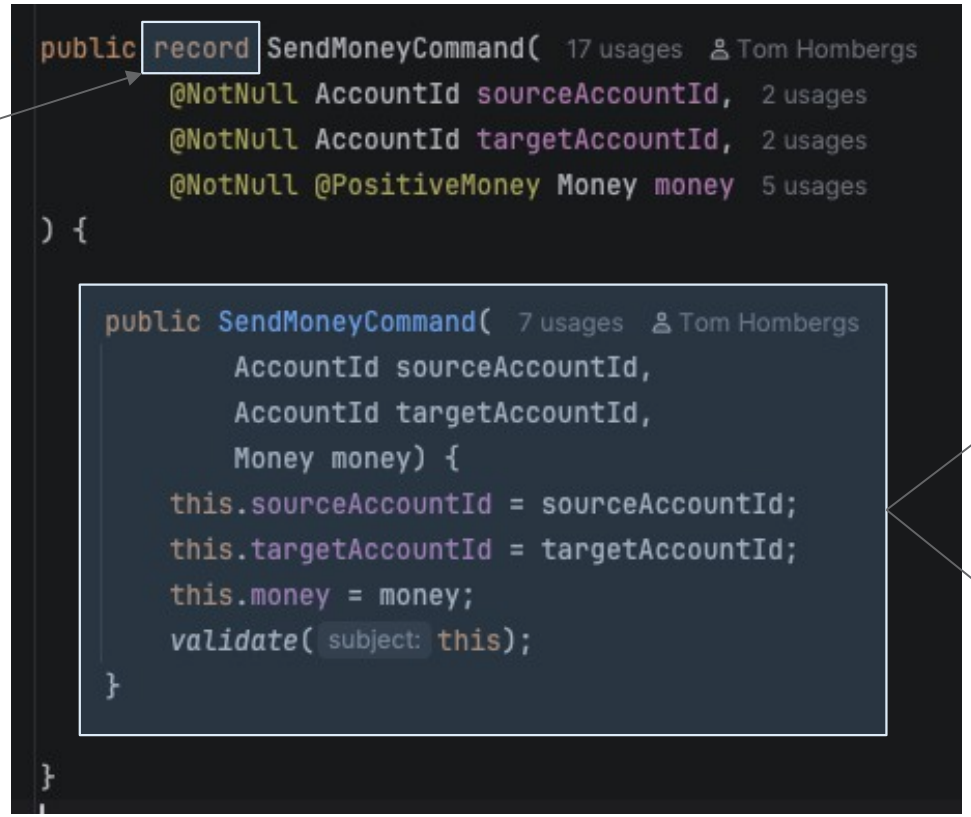
    boolean sendMoney(SendMoneyCommand command);  4 usages  1 implementation  Tom Hombergs

}
```

- SendMoneyUseCase defines **external API**
- An adapter outside of our application core can now invoke this use case by calling sendMoney().
- The SendMoneyController (REST Controller) from the previous slide is one adapter invoking the use case. An alternative could be, e.g., an XMLRPC Controller invoking the use case.

Example of a Command

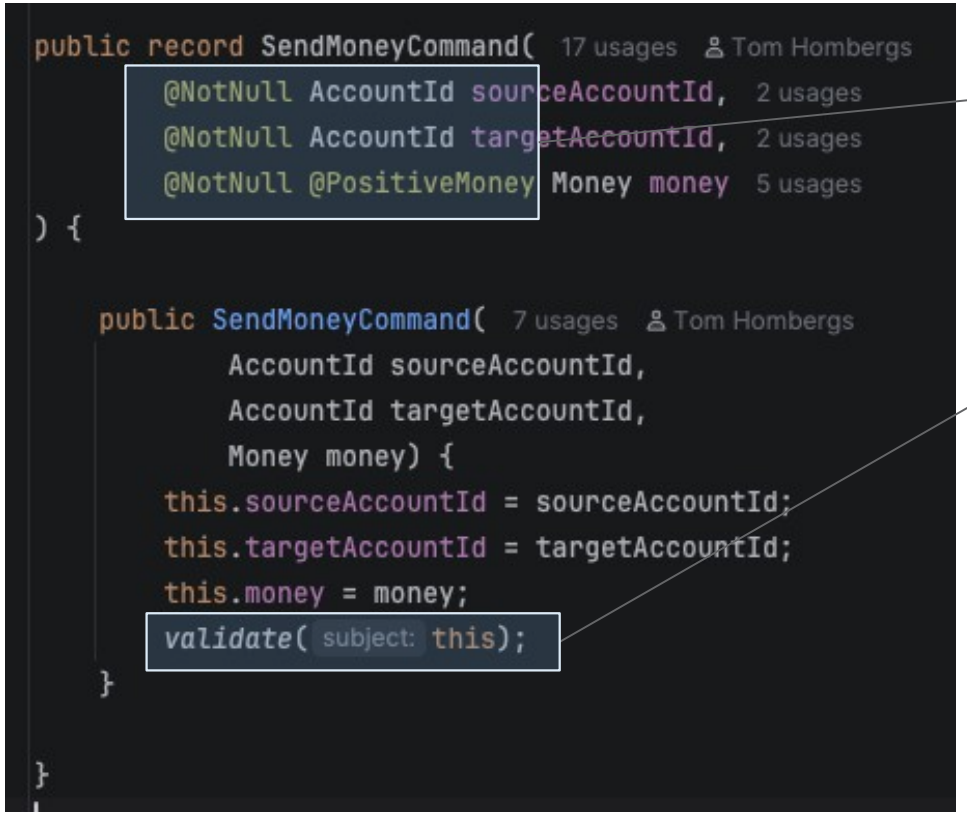
indicates that SendMoneyCommand is a value object and thus is immutable (Java Records are suitable for VOs)



- To transfer money, we need IDs of the source and target account and the amount of money that is to be transferred
- All parameters needed are aggregated into the SendMoneyCommand
- SendMoneyCommand is a value object
- This enables validation in the constructor of the value object

Once constructed successfully, we can be sure that the state is valid and cannot be changed to something invalid

Example of a Command



- Bean annotation are used for validation purposes (de-facto standard in Java)
- Validation in the constructor of the command; in case of a violation an exception is thrown
- This essentially creates an **anti-corruption-layer** around the use case
- Note: Input validations are *syntactic validations*. Syntactic validations can be done without access to the domain. In turn, business rules (invariants) constitute *semantic validations* that require access to the domain.

Implementation of Self Validation

```java
public class Validation {  1 usage  Tom Hombergs

  // Your IDE may complain that the ValidatorFactory needs to be closed, b
  // we break the contract of ValidatorFactory#close.
  private final static Validator validator =  1 usage
          buildDefaultValidatorFactory().getValidator();

  /**
   * Evaluates all Bean Validation annotations on the subject.
   */
  public static <T> void validate(T subject) {  2 usages  Tom Hombergs
    Set<ConstraintViolation<T>> violations = validator.validate(subject);
    if (!violations.isEmpty()) {
      throw new ConstraintViolationException(violations);
    }
  }
}
```

- In case of a violation an exception is thrown

Example of an Application Service

```java
@RequiredArgsConstructor  Tom Hombergs *
@UseCase
@Transactional
public class SendMoneyService implements SendMoneyUseCase {

    private final LoadAccountPort loadAccountPort;
    private final AccountLock accountLock;
    private final UpdateAccountStatePort updateAccountStatePort;
    private final MoneyTransferProperties moneyTransferProperties;

    @Override  4 usages  Tom Hombergs *
    public boolean sendMoney(SendMoneyCommand command) {

        // See source code for full example
        // 1) Validate business rules
        // 2) Update model state
        // 3) Return output

        return true;
    }
```

Implements the incoming port interface *SendMoneyUseCase* (invoked from a controller via a port interface)

Calls the outgoing port interface *LoadAccountPort* to load an account from the repository and the port interface *UpdateAccountStatePort* to persist the updated model state

@Transactional indicates that all public methods inside this class run inside a database transaction

- Transactions should span all write operations to a database that are performed within a use case.
- All operations can be rolled back together if one fails
- Persistence Adapter do not know which other database operations are part of the same use case and thus do not know when to open and close the transaction => delegation of responsibility to service

Example Flow

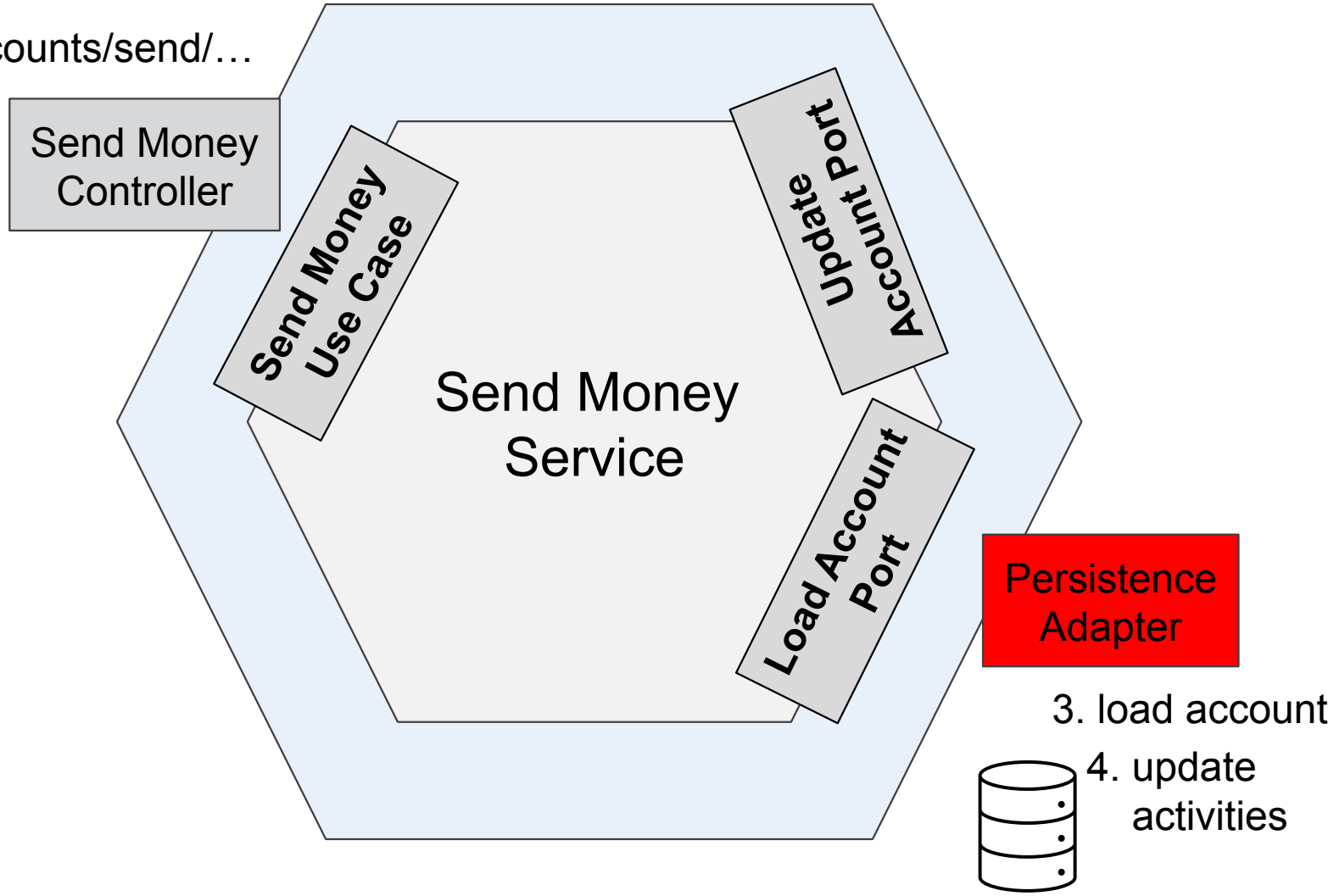
1. send money





Web UI
(frontend)

2. POST: accounts/send/…



3. load account

4. update activities

Example Repository

```java
import org.springframework.data.jpa.repository.JpaRepository;

interface SpringDataAccountRepository extends JpaRepository<AccountJpaEntity, Long> {  1 usage  Tom Hombergs
}
```

Using Spring Data basic CRUD interfaces can be created out of the box.

```java
interface ActivityRepository extends JpaRepository<ActivityJpaEntity, Long> {

    @Query("""  1 usage  Tom Hombergs
            select a from ActivityJpaEntity a
            where a.ownerAccountId = :ownerAccountId
            and a.timestamp >= :since
            """)
    List<ActivityJpaEntity> findByOwnerSince(
            @Param("ownerAccountId") long ownerAccountId,
            @Param("since") LocalDateTime since);

    @Query("""  1 usage  Tom Hombergs
            select sum(a.amount) from ActivityJpaEntity a
            where a.targetAccountId = :accountId
            and a.ownerAccountId = :accountId
            and a.timestamp < :until
            """)
    Optional<Long> getDepositBalanceUntil(
            @Param("accountId") long accountId,
            @Param("until") LocalDateTime until);

    @Query("""  1 usage  Tom Hombergs
            select sum(a.amount) from ActivityJpaEntity a
            where a.sourceAccountId = :accountId
            and a.ownerAccountId = :accountId
            and a.timestamp < :until
            """)
    Optional<Long> getWithdrawalBalanceUntil(
            @Param("accountId") long accountId,
            @Param("until") LocalDateTime until);

}
```

Special interfaces (beyond basic CRUD) can also be implemented, e.g.,
by providing SQL statements in Spring Data JPA

Different DB technology
→ different different type of repository,
e.g., *MongoRepository*

Example Persistence Adapter

Load `Account` via the `AccountRepository` and load the `Activities` for this account for a certain time window.

Creating a valid account also needs the balance the account had before starting this activity window.

Mapping of this data to the Account domain entity and return it to the caller.

Different DB technology
→ new persistence adapter

```
@RequiredArgsConstructor  Tom Hombergs
@PersistenceAdapter
class AccountPersistenceAdapter implements
        LoadAccountPort,
        UpdateAccountStatePort {

    private final SpringDataAccountRepository accountRepository;
    private final ActivityRepository activityRepository;
    private final AccountMapper accountMapper;

    @Override  7 usages  Tom Hombergs
    public Account loadAccount(
                    AccountId accountId,
                    LocalDateTime baselineDate) {

        AccountJpaEntity account =
                accountRepository.findById(accountId.getValue())
                        .orElseThrow(EntityNotFoundException::new);

        List<ActivityJpaEntity> activities =
                activityRepository.findByOwnerSince(
                        accountId.getValue(),
                        baselineDate);

        Long withdrawalBalance = activityRepository
                .getWithdrawalBalanceUntil(
                        accountId.getValue(),
                        baselineDate)
                .orElse( other: 0L);

        Long depositBalance = activityRepository
                .getDepositBalanceUntil(
                        accountId.getValue(),
                        baselineDate)
                .orElse( other: 0L);

        return accountMapper.mapToDomainEntity(
                account,
                activities,
                withdrawalBalance,
                depositBalance);

    }
```

The persistence adapter implements two ports: `LoadAccountPort` and `UpdateAccountStatePort`.

For updating the state of an account, we iterate over all activities in the account and check if they have IDs. If not, they are new activities, that need to be persisted via the repository.

```
    @Override  4 usages  Tom Hombergs
    public void updateActivities(Account account) {
        for (Activity activity : account.getActivityWindow().getActivities()) {
            if (activity.getId() == null) {
                activityRepository.save(accountMapper.mapToJpaEntity(activity));
            }
        }
    }

}
```

Example of a Read-only Use Case

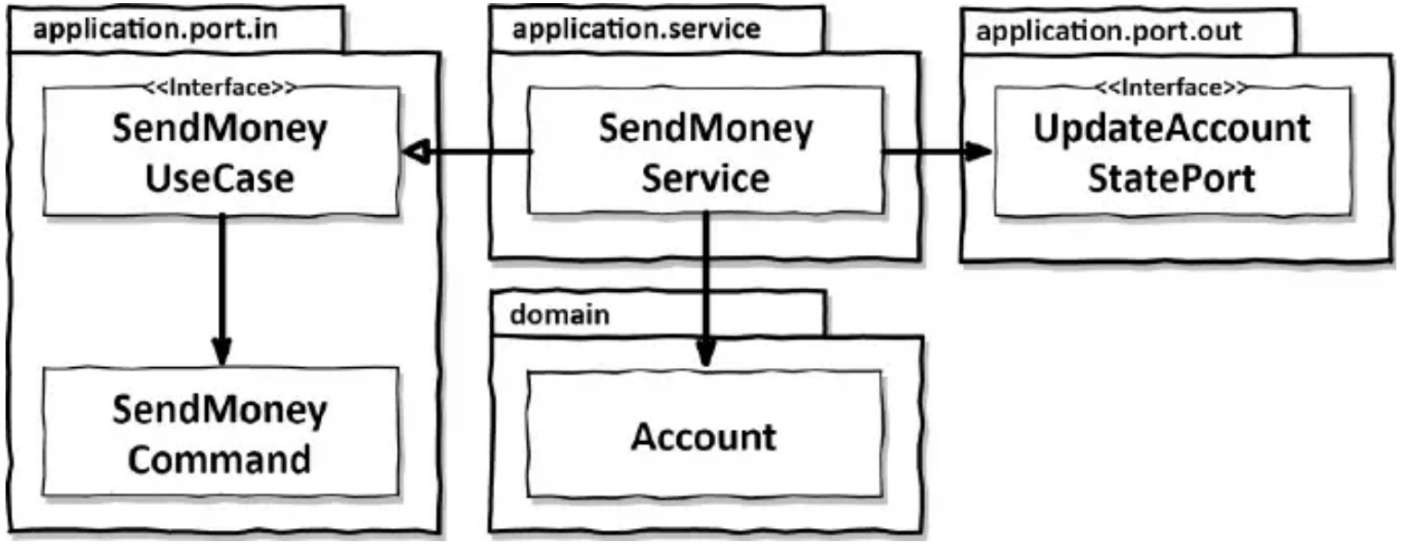
```java
@RequiredArgsConstructor  no usages  Tom Hombergs
class GetAccountBalanceService implements GetAccountBalanceUseCase {

    private final LoadAccountPort loadAccountPort;

    @Override  no usages  Tom Hombergs
    public Money getAccountBalance(GetAccountBalanceQuery query) {
        return loadAccountPort.loadAccount(query.accountId(), LocalDateTime.now())
                .calculateBalance();
    }
}
```

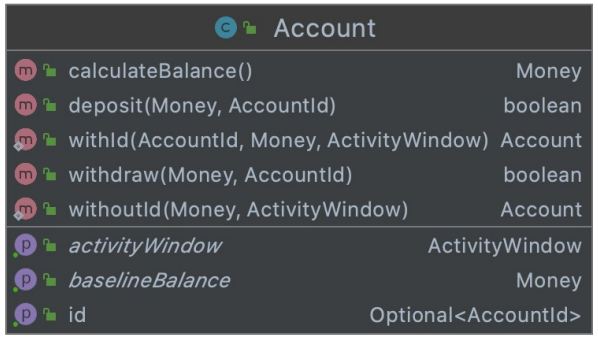
- Implemented as query (vs. command)
- Query Service does not do any work apart from passing the query to the outgoing port

Example Domain Layer



Account Domain

Entity

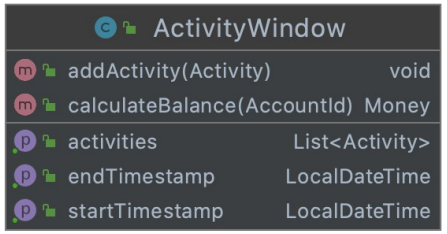


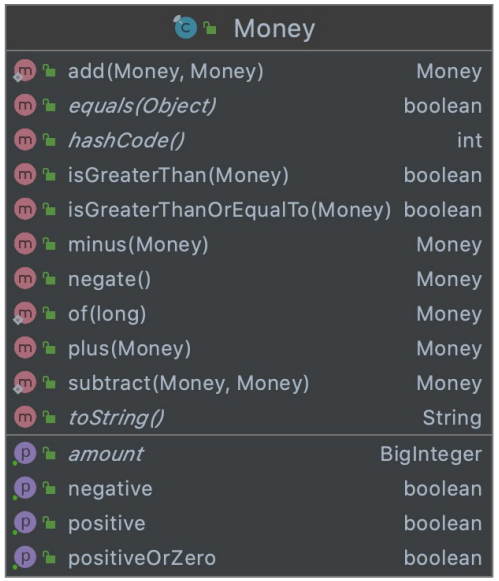
Aggregate

Manages deposits and withdrawals to an account

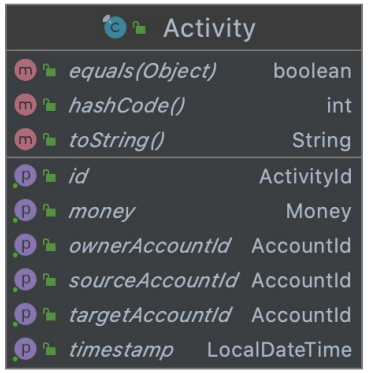
All references from outside go to the aggregate

Value Objects





Entity



Example of an Aggregate

```java
@AllArgsConstructor(access = AccessLevel.PRIVATE)  Tom Hombergs
public class Account {

    /**
     * The unique ID of the account.
     */
    private final AccountId id;

    /**
     * The baseline balance of the account. This was the balance of the account before the first
     * activity in the activityWindow.
     */
    @Getter private final Money baselineBalance;

    /**
     * The window of latest activities on this account.
     */
    @Getter private final ActivityWindow activityWindow;

    /**
     * Creates an {@link Account} entity without an ID. Use to create a new entity that is not yet
     * persisted.
     */
    public static Account withoutId(  no usages  Tom Hombergs
                    Money baselineBalance,
                    ActivityWindow activityWindow) {
        return new Account( id: null, baselineBalance, activityWindow);
    }

    /**
     * Creates an {@link Account} entity with an ID. Use to reconstitute a persisted entity.
     */
    public static Account withId(  2 usages  Tom Hombergs
                    AccountId accountId,
                    Money baselineBalance,
                    ActivityWindow activityWindow) {
        return new Account(accountId, baselineBalance, activityWindow);
    }

    public Optional<AccountId> getId() { return Optional.ofNullable(this.id); }
```

```java
public boolean withdraw(Money money, AccountId targetAccountId) {

    if (!mayWithdraw(money)) {
        return false;
    }

    Activity withdrawal = new Activity(
            this.id,
            this.id,
            targetAccountId,
            LocalDateTime.now(),
            money);
    this.activityWindow.addActivity(withdrawal);
    return true;
}

private boolean mayWithdraw(Money money) {  1 usage  Tom Hombergs
    return Money.add(
            this.calculateBalance(),
            money.negate())
            .isPositiveOrZero();
}

/**
 * Tries to deposit a certain amount of money to this account.
 * If sucessful, creates a new activity with a positive value.
 * @return true if the deposit was successful, false if not.
 */
public boolean deposit(Money money, AccountId sourceAccountId) {
    Activity deposit = new Activity(
            this.id,
            sourceAccountId,
            this.id,
            LocalDateTime.now(),
            money);
    this.activityWindow.addActivity(deposit);
    return true;
}
```

Before withdrawing money, the business rules (i.e., ***semantic validation***) are checked to ensure that we cannot overdraw an account.

swa Software Architecture Teaching and Research Unit | RWTH AACHEN UNIVERSITY

Example of a Value Object

Alternative to Java Records: @Value annotation from Project Lombok

```java
@Value  Tom Hombergs
public class Money {

    public static Money ZERO = Money.of( value: 0L);

    @NonNull
    private final BigInteger amount;

    public boolean isPositiveOrZero() { return this.amount.compareTo(BigInteger.ZERO) >= 0; }

    public boolean isNegative() { return this.amount.compareTo(BigInteger.ZERO) < 0; }

    public boolean isPositive() { return this.amount.compareTo(BigInteger.ZERO) > 0; }

    public boolean isGreaterThanOrEqualTo(Money money) { return this.amount.compareTo(money.amount) >= 0; }

    public boolean isGreaterThan(Money money) { return this.amount.compareTo(money.amount) >= 1; }

    public static Money of(long value) { return new Money(BigInteger.valueOf(value)); }

    public static Money add(Money a, Money b) { return new Money(a.amount.add(b.amount)); }

    public Money minus(Money money) { return new Money(this.amount.subtract(money.amount)); }

    public Money plus(Money money) { return new Money(this.amount.add(money.amount)); }

    public static Money subtract(Money a, Money b) { return new Money(a.amount.subtract(b.amount)); }

    public Money negate() { return new Money(this.amount.negate()); }

}
```

Addition: Emitting an Integration Event

Note: not part of the original Buckpal Example

Domain Events:

- Represent a change in the state of one or more domain objects (**Aggregates**)
- Used to facilitate communication and coordination between different parts of the system.
- Domain Events which are sent to external processes (out-of-process communication) are often referred to as Integration Events

Addition: Example of Emitting an Integration Event (1)

```java
@RequiredArgsConstructor
@UseCase
@Transactional
public class SendMoneyService implements SendMoneyUseCase {

    private final MoneySentEventPort moneySentEventPort;
    private final MoneyTransferProperties moneyTransferProperties;

    @Override
    public boolean sendMoney(SendMoneyCommand command) {

        //TODO validate business rules
        //TODO update model state
        //TODO return output

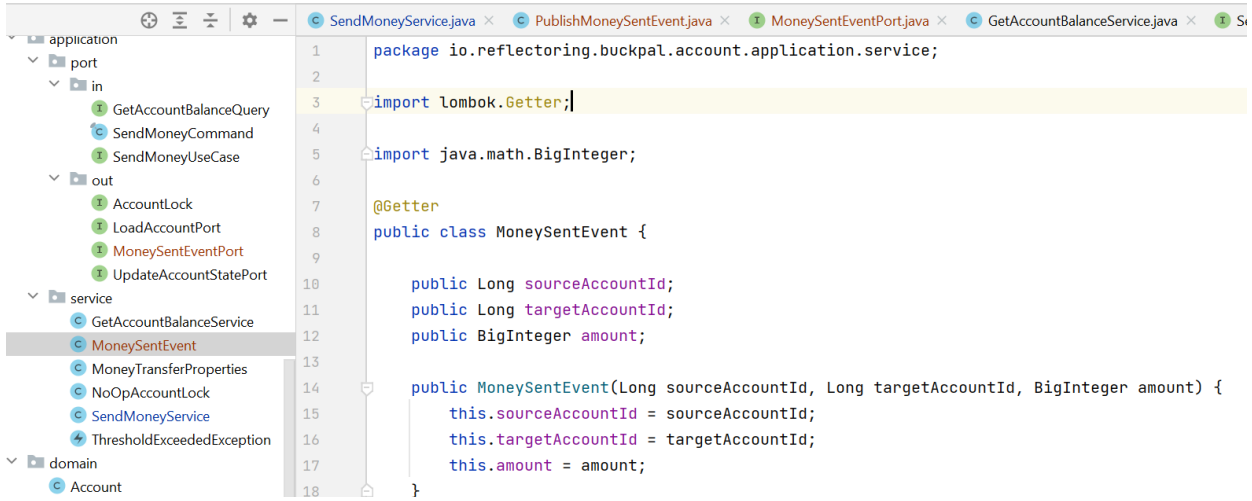
        MoneySentEvent moneySentEvent = new MoneySentEvent(command.getSourceAccountId().getValue(),
                command.getTargetAccountId().getValue(), command.getMoney().getAmount());
        moneySentEventPort.publishMoneySentEvent(moneySentEvent);

        return true;
    }
}
```

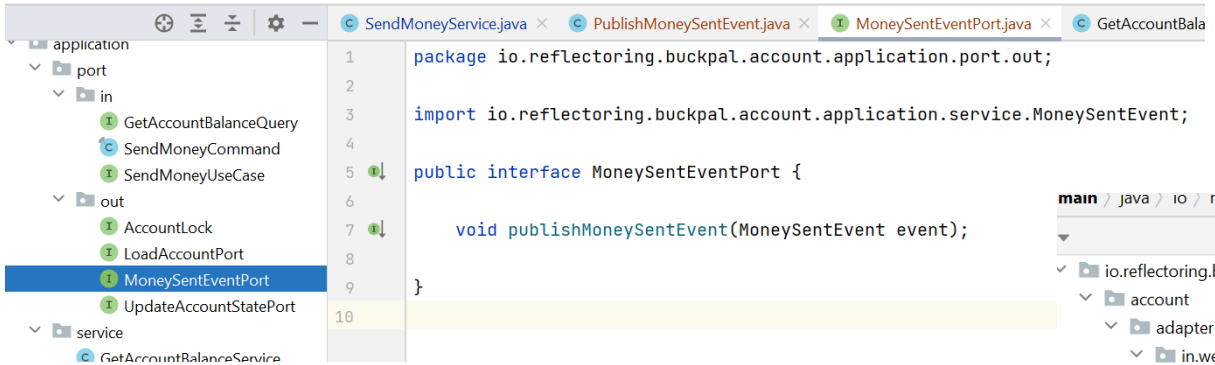
- Application services emit events to the outside of the bounded context (e.g., to other micro-services or message brokers)
- Using an out-going (driven) port from the application service
- Port can be implemented by different adapters (e.g., for using different communication forms such as Kafka or MQTT)
- Inter-service communication will be discussed next week

- Integration events should only be emitted when the use case finished successfully

Addition: Example of Emitting an Integration Event (2)

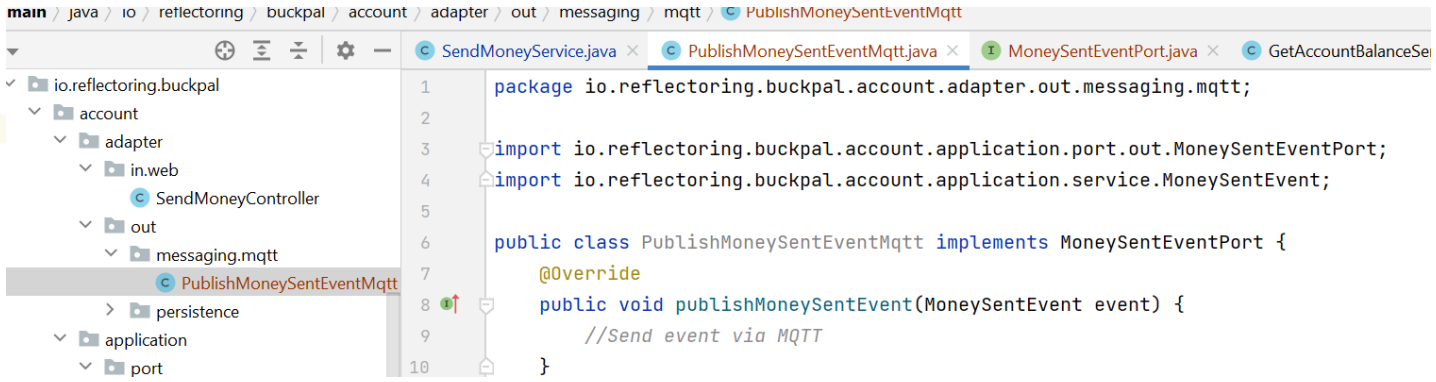


Integration event used in application service

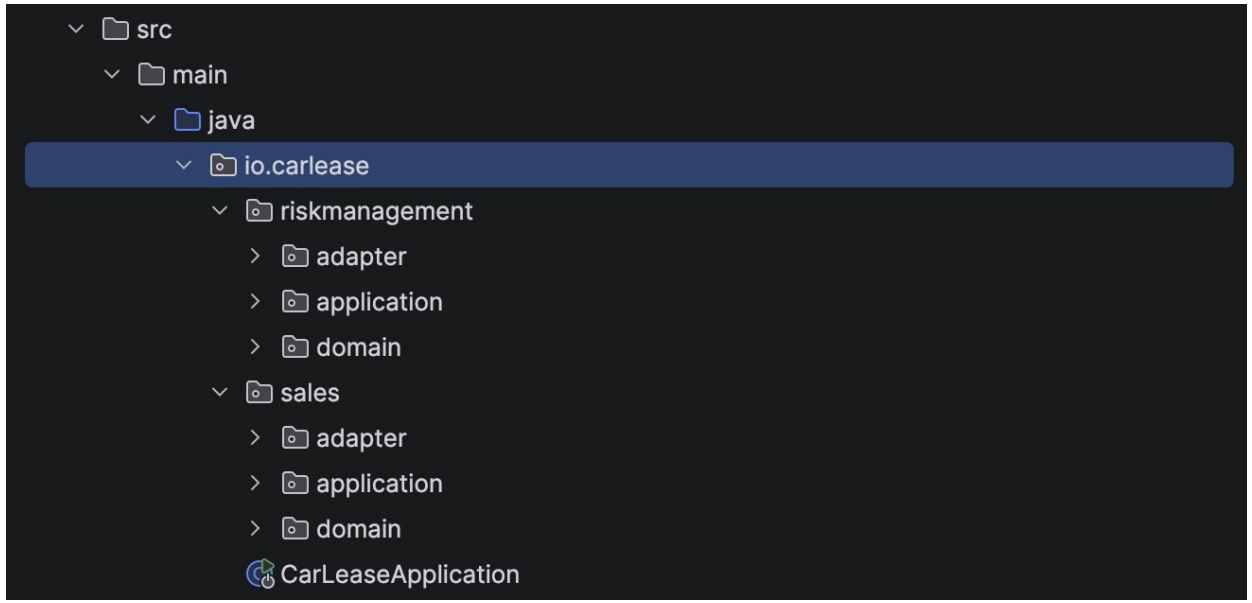


Out-going port in application layer used in application service

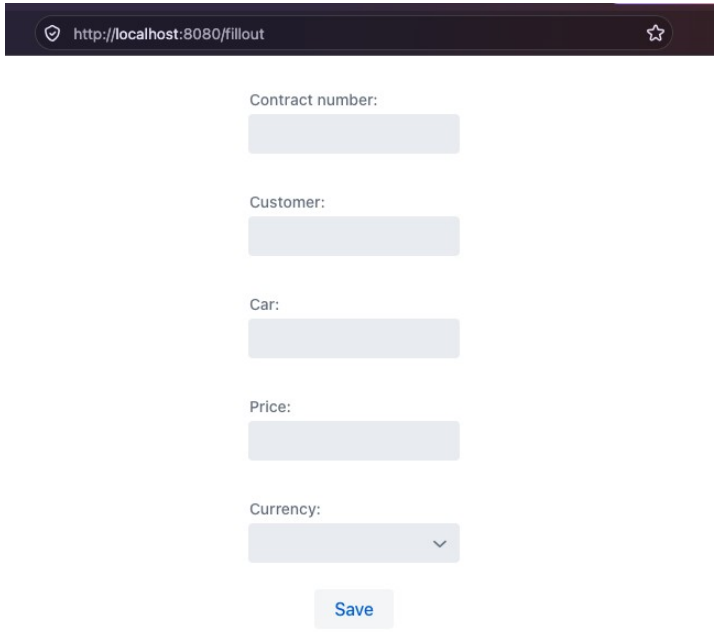
Adapter implementing the port



Car Lease Project: A Modular Monolith

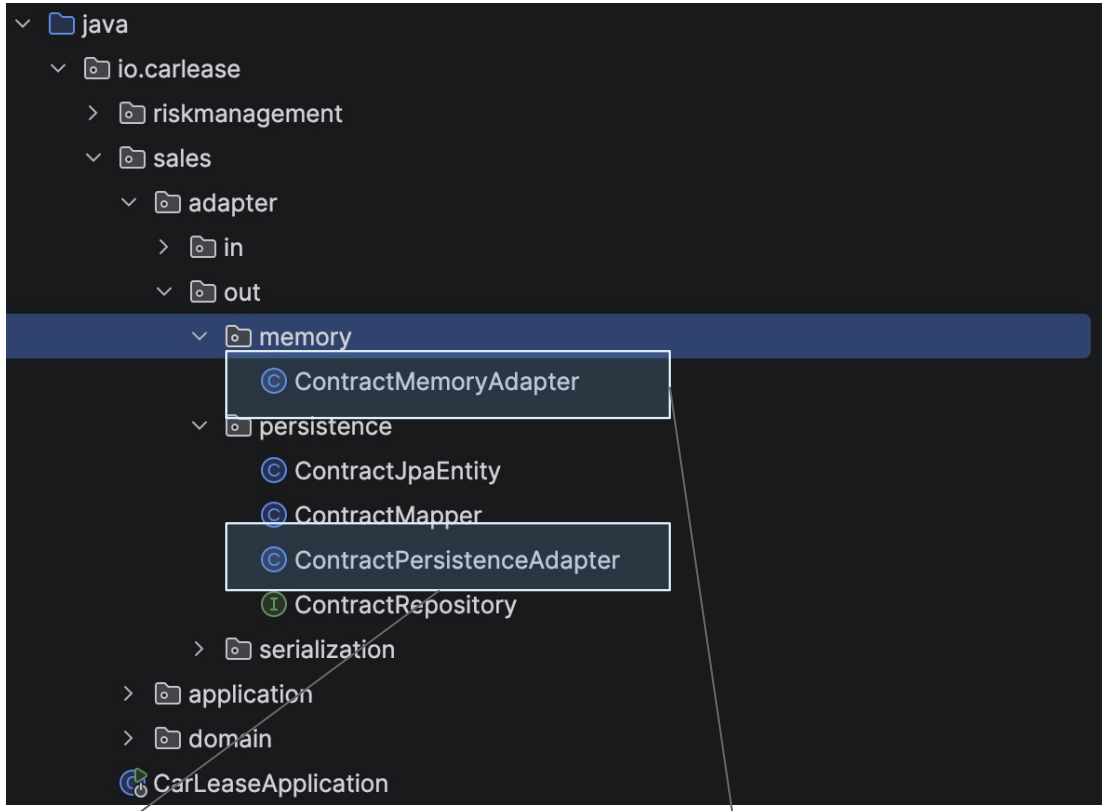


- Two bounded contexts; each implemented as a hexagon
- Spring Boot application, with a Vaadin-based (web) front-end
- Shows a web form at: http://localhost:8080/fillout



https://gitlab.git.nrw/rwth-swa/teaching/carlease

Switching Persistence Adapters



Saving a contract in a h2 db in memory via Hibernate

Saving a contract in memory

Two persistence adapters that implement the same port; both are declared as @Components
→ Spring will complain, it does not know which one to use

```
@Component  marco
public class ContractMemoryAdapter implements ContractPersistencePort {
```

```
@Component  marco
@Primary
public class ContractPersistenceAdapter implements ContractPersistencePort {
```

Declare component (adapter) as @Primary
→ Easily switch adapters (one line)

Switching Persistence Adapters in Carlease

Output after filling out the web form, with using the InMemoryAdapter

```
04-11-2026 10:16:36.720 [http-nio-8080-exec-1] INFO  org.apache.catalina.core.ContainerBase.[Tomcat].[localhost].[/].log - Initializing Spring DispatcherServlet
04-11-2026 10:16:36.720 [http-nio-8080-exec-1] INFO  org.springframework.web.servlet.DispatcherServlet.initServletBean - Initializing Servlet 'dispatcherServlet
04-11-2026 10:16:36.721 [http-nio-8080-exec-1] INFO  org.springframework.web.servlet.DispatcherServlet.initServletBean - Completed initialization in 1 ms
04-11-2026 10:16:41.043 [http-nio-8080-exec-2] INFO  com.vaadin.flow.spring.SpringInstantiator.getI18NProvider - The number of beans implementing 'I18NProvider'
04-11-2026 10:16:46.800 [http-nio-8080-exec-7] INFO  io.carlease.sales.application.service.FilloutContractService.with - A contract was persisted with number 1
```

Output after filling out the web form, with using the PersistenceAdapter

```
04-11-2026 10:01:21.310 [http-nio-8080-exec-1] INFO  org.springframework.web.servlet.DispatcherServlet.initServletBean - Completed initialization in 1 ms
04-11-2026 10:01:21.343 [http-nio-8080-exec-1] INFO  com.vaadin.flow.spring.SpringInstantiator.getI18NProvider - The number of beans implementing 'I18NProvider'
Hibernate: select cje1_0.number,cje1_0.approval_date,cje1_0.car,cje1_0.interest_per_year,cje1_0.lease_term_in_months,cje1_0.lessee,cje1_0.price_amount,cje1_0.pri
Hibernate: select cje1_0.number,cje1_0.approval_date,cje1_0.car,cje1_0.interest_per_year,cje1_0.lease_term_in_months,cje1_0.lessee,cje1_0.price_amount,cje1_0.pri
Hibernate: insert into contracts (approval_date,car,interest_per_year,lease_term_in_months,lessee,price_amount,price_currency,number) values (?,?,?,?,?,?,?,?)
04-11-2026 10:12:03.776 [http-nio-8080-exec-1] INFO  io.carlease.sales.application.service.FilloutContractService.with - A contract was persisted with number 12
```

Hibernate showing the SQL statements